DOM Events in JavaScript

Event Listeners, Event Objects, and Interactivity

1. Approaches to Adding Events

Events allow JavaScript to react to user interactions (clicks, keyboard presses, mouse movements). There are three primary ways to attach events, but only one is recommended for modern development.

- **Inline HTML Events (Avoid):** Writing `onclick="function()"` directly inside the HTML tag. This is bad practice because it mixes UI structure with logic.
- **Direct Properties (Limited):** Writing `element.onclick = function() {}` in JavaScript. This is better, but it has a major flaw: **it overwrites any previous events** attached to that element.
- `addEventListener()` **(Best Practice):** The modern standard. It allows you to attach multiple listeners to a single element without overwriting, and gives you control over the propagation phase.

2. The Event Object (`e`)

When an event occurs, the browser automatically passes an **Event Object** (usually written as `e` or `event`) into your callback function. This object contains all the data about what just happened.

Property	What it tells you
`e.type`	The type of event that was triggered (e.g., "click", "keydown", "mouseover").
`e.target`	The exact, specific HTML element that was physically interacted with (e.g., the specific image inside a list).
`e.currentTarget`	The element that the event listener is actually attached to.
`e.clientX / Y`	The X and Y coordinates of the mouse pointer relative to the visible browser window (viewport) at the time of the event.
`e.altKey / ctrlKey`	Boolean values (true/false) indicating if the Alt, Ctrl, or Shift keys were being held down while the event occurred.

3. Event Propagation: Bubbling vs. Capturing

When you click an element nested deeply inside other elements (like an `<img>` inside an `<li>` inside a `<ul>`), the browser needs to figure out which element's click event to fire first. This is called Event Propagation.

```
      // Syntax: addEventListener(event, callback, useCapture) // Event Bubbling (Default: false) document.getElementById('images').addEventListener('click', function(e){ console.log("Clicked inside the UL"); }, false);
```

Event Bubbling (Inside-Out): This is the default behavior (the third argument is `false` or omitted). The event triggers on the innermost target element first, and then "bubbles up" to its parents. If you click an image, it triggers the image's event, then the li's event, then the ul's event.

Event Capturing (Outside-In): If you pass `true` as the third argument, the event is caught at the top-most parent first, and trickles down to the target element.

> 💡 **Tutor's Note: When to use which?**
> 95% of the time in modern web development, you will use **Event Bubbling** (default). Event Capturing is a legacy concept originally introduced to resolve browser compatibility issues between old Netscape and Internet Explorer models. However, Capturing can occasionally be useful in highly complex UI architectures where a parent container strictly needs to intercept and block events before they reach child components.

4. Stopping Propagation

Sometimes you don't want an event to bubble up. If you click a child element, you might want to prevent the parent container's click event from firing.

```
      document.getElementById('owl-image').addEventListener('click', function(e){ console.log("Owl Clicked"); e.stopPropagation(); // Stops the event from bubbling up to parents });
```

5. Preventing Default Behaviors

Many HTML elements have built-in default behaviors. For example, clicking an `<a>` tag redirects the page, and submitting a `<form>` refreshes the page. You can halt these built-in actions using `e.preventDefault()`.

```
document.getElementById('google-link').addEventListener('click', function(e){
e.preventDefault(); // The browser will no longer navigate to the link
console.log("Navigation blocked."); });
```

6. Practical Project: Event Delegation

Goal: Clicking on any image in a gallery should remove it from the page.

Bad Approach: Adding a separate `addEventListener` to every single image. If you have 1,000 images, that's 1,000 listeners clogging the memory.
Optimized Approach (Event Delegation): Add *one* listener to the parent `<ul>`, and use `e.target` to identify which image bubbled the click up.

```
document.querySelector('#image-gallery').addEventListener('click', function(e){
// e.target is the exact element clicked (the img) // e.target.parentNode is the
wrapper (the li) // Strict Check: Ensure we only remove if an IMG was actually clicked
if(e.target.tagName === 'IMG'){ let removeIt = e.target.parentNode; removeIt.remove();
} }, false);
```

⚠️ Tutor's Edge Case: The tagName Check
Why do we need `if(e.target.tagName === 'IMG')`? If you click on the padding of the `<ul>` (outside the images), `e.target` becomes the `<ul>` itself. Without the check, `e.target.parentNode.remove()` would delete the `<body>` or main container, erasing your entire web page content in one click! Always verify the element's identity before performing destructive DOM operations.